2025-2026 学年第一学期

武汉大学测绘学院导航工程专业

5G 实验 2：室内指纹定位

课程名称： 位置服务与实践

班　　级： 测绘 6 班

姓　　名： 刘峻琰

学　　号： 2023302141249

实验名称： 室内指纹定位

指导教师： 陈亮

实验时间： 2026 年 5 月 6 日

目录

一、 实践目的 3

二、 实践原理 3

2.1 5G 指纹定位基本原理 3

2.2 定位特征 CSI 与 EDP 4

2.3 加权 K 近邻 (WKNN) 定位算法 5

三、 实验环境与数据说明 6

3.1 实验环境 6

3.2 测试场景 6

3.3 数据集结构 7

四、 实验步骤 7

4.1 数据加载与结构验证 7

4.2 特征提取与预处理 8

4.3 数据标准化 8

4.4 交叉验证选择最优参数 9

4.5 WKNN 定位计算 9

4.6 结果保存与可视化 10

五、 结果与分析 10

5.1 数据预处理结果 10

5.2 交叉验证参数优化结果 11

5.3 最终定位结果 11

六、 结论 13

七、 程序代码 13

一、实践目的

本实验基于武汉大学测绘遥感信息工程国家重点实验室主楼采集的真实 5G 信号数据，设计并实现一套完整的室内指纹定位系统。具体目标如下：

(1) 深入理解 5G 室内指纹定位的基本原理，掌握离线指纹数据库构建与在线定位匹配的完整技术流程。

(2) 学习并实践 5G 信道状态信息 (CSI) 和能量延迟剖面 (EDP) 两类典型信号特征的物理意义、提取方法与预处理技术。

(3) 实现加权 K 近邻 (WKNN) 定位算法，并通过交叉验证方法科学地确定最优近邻数 K，保证定位精度与泛化能力。

(4) 对定位结果进行定量与定性分析，评价算法的精度、稳定性和鲁棒性，验证 5G 指纹定位在真实室内场景下的可行性与优势。

(5) 通过 MATLAB 编程全流程实践，提升大规模信号数据处理、统计机器学习算法实现与实验可视化分析的综合能力。

二、实践原理

2.1 5G 指纹定位基本原理

5G 指纹定位是一种基于无线信号空间特征与地理位置映射关系的室内定位方法，其核心思想是：无线信号在室内环境中经过直射、反射、散射等多径传播后，形成的信道状态蕴含了丰富且独特的位置相关信息，如同每个位置的“指纹”。整个定位过程分为两大阶段：

1. 离线建库阶段（训练）
   (1) 在目标区域内布设一定数量的参考点（Reference Point, RP），记录每个参考点的真实坐标 pi。
   (2) 在每个参考点处多次采集 5G 信号，提取能够表征信道状态的特征向量 fi，典型特征包括 CSI 和 EDP。
   (3) 将所有参考点的“位置-特征”对构成指纹数据库 $\mathrm{D} = \{(\mathrm{p_i}, \mathrm{f_i})\}_{\mathrm{i=1}}^{\mathrm{N}}$ 。
2. 在线定位阶段（测试）

(1) 在未知位置采集相同的信号特征，得到查询特征向量 fq。

(2) 通过某种相似度度量（如欧氏距离、余弦相似度等）在指纹库中找到与 fq 最匹配的若干个参考点。

(3) 利用这些参考点的位置信息，通过插值或加权方法估计未知位置的坐标。

相较于传统 WiFi 指纹定位，5G 信号具有更大的带宽、更密集的天线阵列和更精确的信道估计能力，能够提供更高时间分辨率和空间分辨率的信道特征，从而显著提升室内定位精度，有望达到亚米级甚至厘米级水平。

2.2 定位特征 CSI 与 EDP

本实验同时采用两种互补的 5G 信道特征，以增强位置指纹的区分度和鲁棒性。

1. 信道状态信息 (CSI)

CSI 描述了无线信号在发射端与接收端之间每个子载波上的幅度衰减和相位旋转。在 5G 系统中，由于采用了正交频分复用 (OFDM) 技术和波束赋形，CSI 能够精细刻画不同传播路径的复合响应。

本实验所用 CSI 特征的具体结构为：8 个独立波束，每个波束包含 60 个子载波；全部子载波共计 8×60=480 维原始特征；CSI 的幅度对位置变化极为敏感，微小的位置移动可能导致子载波幅度分布的显著改变，因此是指纹定位的核心特征。

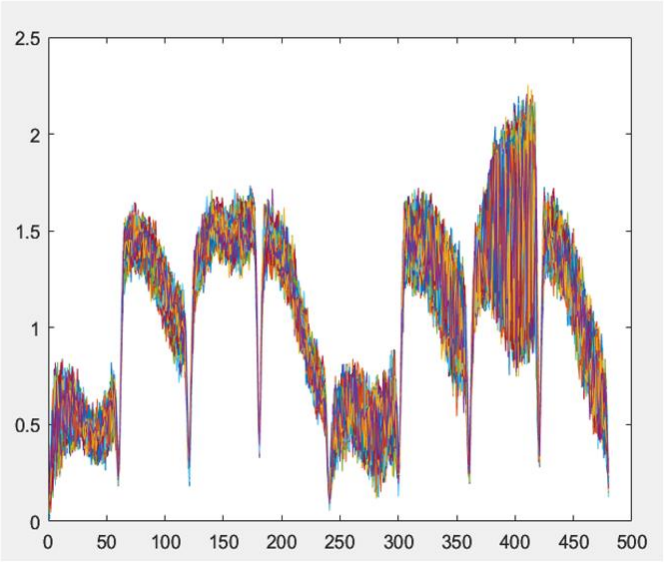


图 1 8 波束 CSI 特征

2. 能量延迟剖面 (EDP)

EDP 表征了接收信号功率在不同时延上的分布情况。它直接反映了多径信号的到达时间和相对强度。

本实验中 EDP 特征的结构为：每个波束提取 1 个能量延迟特征值，8 个波束共 8 维特征；EDP 特征受瞬时噪声干扰较小，具有更好的时间稳定性，能够弥补 CSI 在某些复杂多径环境下的剧烈波动。CSI 与 EDP 两者结合，能够从“频域细节”和“时域能量分布”两个维度共同描述信道的空间特性，大幅提高定位的准确性与可靠性。

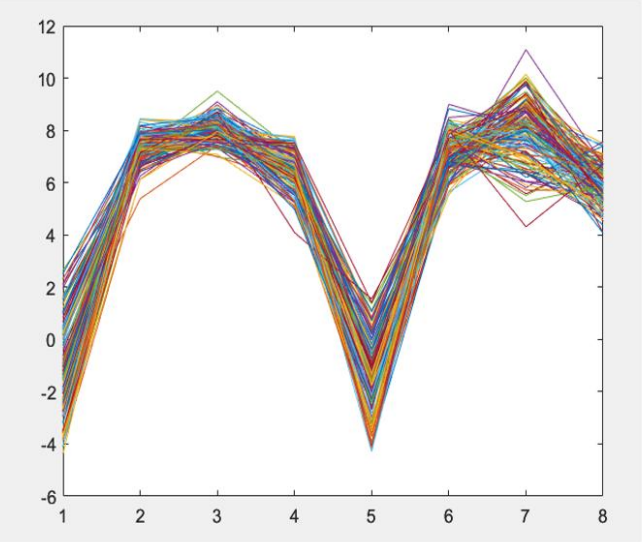


图 2 8 波束 EDP 特征

2.3 加权 K 近邻 (WKNN) 定位算法

K 近邻 (KNN) 算法简单、高效，是指纹定位中最常用的匹配方法之一。然而，标准的 KNN 对所有邻居赋予相同权重，忽视了距离远近带来的信息可靠性差异。WKNN 对此进行改进，依据邻居的相似度赋予不同的贡献权重。本实验采用基于高斯核的 WKNN 算法，具体步骤如下：

1. 计算距离

   对查询特征向量 fq 与指纹库中每个参考点的特征向量 fi，计算欧氏距离（或加权欧氏距离）：

$$d_i = \left\| f_q - f_i \right\|_2 \tag{1}$$

2. 选择邻居

将所有距离升序排列，选取距离最小的 K 个参考点作为候选邻居集合 NK。

3. 计算高斯权重

令σ为 K 个邻居距离的中位数，或直接取其标准差。第 i 个邻居的权重定义为：

$$w_i = \exp\left(-\frac{d_i^2}{2\sigma^2}\right) \tag{2}$$

高斯核函数的优势在于：对距离较小的点赋予接近 1 的权重，而对距离较大的点权重指数衰减，既突出最相似指纹的作用，又避免噪声点被过度放大。

4. 加权估计位置

将各邻居的坐标按其权重进行归一化加权平均，得到最终预测位置：

$$\hat{p} = \frac{\sum_{i\in\mathcal{N}_K} w_i\, p_i}{\sum_{i\in\mathcal{N}_K} w_i} \tag{3}$$

与倒数加权、均匀加权等方案相比，高斯加权对异常距离值具有更好的抑制作用，能够提高在信号波动情况下的定位稳定性。

三、实验环境与数据说明

3.1 实验环境

本实验的编程与计算环境基于 MATLAB R2023b 平台搭建，依托 MATLAB 强大的矩阵运算能力、内置的统计分析与科学可视化工具完成 5G 信号特征的提取、预处理与定位算法的实现，同时调用统计与机器学习工具箱实现 Z-score 标准化、欧氏距离批量计算与留一法交叉验证等核心功能，所有实验数据均采用 MATLAB 原生的.mat 格式进行存储与读取，保证了大规模信号数据处理的高效性与数据格式的兼容性。

3.2 测试场景

测试场地选在武汉大学测绘遥感信息工程国家重点实验室主楼四楼休闲厅，该区域具有典型的室内多径环境，包括墙壁、玻璃窗、家具等障碍物，能够真实反映 5G 信号在室内传播的特征。

场地内均匀部署了 12 个参考点，形成规则的 3 行 × 4 列网格：

X 轴方向（横向）：0 m、1 m、2 m、3 m，间隔 1 m；

Y 轴方向（纵向）：0 m、4 m、8 m，间隔 4 m。

此网格覆盖了 3m × 8m 的矩形区域，为后续定位提供了均匀、完整的空间指纹覆盖。测试点则分布在该区域内，具体坐标未在数据中给出，由算法预测。

3.3 数据集结构

实验提供两个独立的 MATLAB 数据文件，数据结构清晰规范。

1. 训练数据 (train_data.mat)

内含一个结构体数组，长度为 12，对应 12 个参考点。每个结构体包含三个字段：

- coor：1×2 浮点数向量，表示参考点的真实二维坐标 [x, y]（单位：米）；
- csi：1×1 cell 数组，内部存储 144×480 的双精度矩阵，144 为样本数，480 为 CSI 原始特征维度（8 波束 × 60 子载波）；
- csi_EDP：1×1 cell 数组，内部存储 144×8 的双精度矩阵，144 为样本数，8 为 EDP 特征维度。

每个参考点均采集了 144 个独立样本，以覆盖该位置处的信号时间波动。

2. 测试数据 (test_data_no_coor.mat)

内含一个结构体数组，长度为 6，对应 6 个测试点。每个结构体仅包含 csi 和 csi_EDP 两个字段，数据维度与训练数据一致（每个测试点 144 个样本），但不提供坐标信息。测试点的真实位置需由算法预测得到。

四、实验步骤

4.1 数据加载与结构验证

首先使用 MATLAB 的 load 函数分别读入 train_data.mat 和 test_data_no_coor.mat。通过检查工作区变量，确认：

训练集包含 12 个结构体元素，即 12 个参考点；

测试集包含 6 个结构体元素，即 6 个未知点；

每个参考点的 csi 和 csi_EDP 矩阵维度均为 144×480 和 144×8，与预期相符；每个参考点的坐标字段 coor 存在且尺寸为 1×2。此步骤用于保证后续程序不会因数据结构不匹配而出错。

4.2 特征提取与预处理

原始 CSI 特征具有 480 维的高维度，直接使用不仅计算量大，还容易包含大量子载波级别的噪声和冗余信息。因此，本实验设计了一套"降维+融合+过滤"的预处理流水线。

1. CSI 统计特征提取

对每个波束的 60 个子载波，分别计算其幅度值的均值和方差：

- 第 b 个波束的均值：$\mu_b = \frac{1}{60}\sum_{k=1}^{60} |h_{b,k}|$
- 第 b 个波束的方差：$\sigma_b^2 = \frac{1}{60}\sum_{k=1}^{60} (|h_{b,k}| - \mu_b)^2$

由此，原始 480 维 CSI 被压缩为 8 个均值+ 8 个方差，共 16 维。

2. 特征融合

将 16 维 CSI 统计特征与 8 维 EDP 特征直接拼接，形成 24 维的综合特征向量。融合后的特征向量既包含频域的细粒度信息，又包含时延能量分布信息。

3. 异常值过滤

信号采集过程中可能因临时遮挡、硬件抖动等原因产生异常样本。本实验采用四分位数法 (IQR) 进行过滤：

(1) 对每一维特征，计算其第一四分位数Q_1和第三四分位数Q_3，以及四分位距$IQR = Q_3 - Q_1$；

(2) 设定过滤边界：下界$Q_1 - 1.5 \times IQR$，上界$Q_3 + 1.5 \times IQR$；

(3) 只有当某样本的所有 24 维特征均落在对应边界之内时，才将该样本保留。

IQR 法不依赖数据分布假设，对偏态分布和少量异常值具有较好的鲁棒性。经过过滤后，训练集样本数由原始的 $12 \times 144 = 1728$ 降至 1448，剔除了约 16% 的潜在噪声样本，显著提升了指纹库的纯净度。

4.3 数据标准化

不同的特征维度数值范围差异较大，直接计算欧氏距离会导致大数吃小数的问题。因此采用 Z-score 标准化，使每个特征维度均值为 0，标准差为 1：

$$x' = \frac{x-\mu}{\sigma} \tag{4}$$

其中μ和σ均基于训练集计算。测试集则使用与训练集完全相同的μ和σ进行归

一化，保证训练与测试的一致性。对于标准差为 0 的常数特征，将其标准差设为 1 以避免除零错误。

4.4 交叉验证选择最优参数

K 值的选取直接影响 WKNN 的定位性能：K 过小，模型易受单个噪声样本影响；K 过大，则可能引入空间距离较远、信号特征不相似的参考点，导致偏差增大。本实验采用留一法交叉验证 (Leave-One-Out Cross-Validation, LOOCV) 对 K 值进行优化：

1. 将训练集中每个参考点所有样本作为一个整体，依次拿出一个参考点的全部样本作为验证集，其余参考点的样本作为训练子集；
2. 在训练子集上构建指纹库，对验证集中每个样本进行 WKNN 预测，计算预测坐标与真实坐标的欧氏距离；
3. 遍历 K = 35 到 K = 10，记录每个 K 下所有验证样本的平均定位误差。

这种方法充分利用了有限的训练数据，能够较准确地反映模型在未知数据上的表现。

4.5 WKNN 定位计算

选定最优 K 值（K = 5）后，对 6 个测试点分别进行定位预测。每个测试点包含 144 个信号样本，算法对每一个样本均独立进行一次 WKNN 定位，得到 144 个中间坐标估计。

这些中间估计值会因瞬时信号波动而呈现一定散布。为获得一个稳定、可靠的代表性结果，实验采用中位数聚合：对每个测试点的 144 个坐标估计，分别计算 X 坐标分量的中位数和 Y 坐标分量的中位数，以此作为该测试点的最终坐标。中位数对于数据中的离群值具有天然的抗干扰能力，相较于算术平均更为稳健。

在距离度量方面，考虑到 EDP 特征对环境的稳定性优于部分 CSI 统计量，在计算欧氏距离时对 EDP 维度施加了适度的权重放大（实验取权重 2.0），使定位更加依赖稳健特征。

4.6 结果保存与可视化

将 6 个测试点的最终坐标整理为 6×2 的矩阵，保存至 predicted_coordinates.mat 文件中，作为实验提交的标准输出。

为直观展示定位效果，绘制如下可视化图形：

1. 蓝色大圆点：12 个训练参考点的位置；
2. 浅粉色小散点：6 个测试点各自的 144 次单样本预测结果，展示预测分布的聚集程度；
3. 深红色大圆点及数字标注：每个测试点的最终聚合坐标。

图形坐标轴范围限定在场地实际区域 $[-0.5, 3.5] \times [-0.5, 8.5]$，便于目测评价。

五、结果与分析

5.1 数据预处理结果

经过 IQR 异常值过滤后，各参考点和测试点的有效样本数量如下表所示。

表 1 参考点样本过滤统计

参考点编号	原始样本数	过滤后样本数
1	144	124
2	144	127
3	144	123
4	144	127
5	144	119
6	144	126
7	144	128
8	144	115
9	144	112
10	144	122
11	144	120
12	144	109

表 2 测试点样本过滤统计

测试点编号	原始样本数	过滤后样本数
1	144	118
2	144	116
3	144	124
4	144	130
5	144	111
6	144	129

最终，训练集总有效样本数为 1448，特征维度降为 24。从表中可以看出，不同位置受环境多径影响程度不同，样本保留率存在一定差异，但整体有效样本数仍保持在 109 以上，满足后续建模需求。异常值过滤去除了可能由瞬间遮挡或设备抖动导致的失真样本。

5.2 交叉验证参数优化结果

留一法交叉验证得到不同 K 值下的平均定位误差详见表 3。

表 3 不同 K 值的交叉验证平均误差

K 值	平均定位误差(m)
5	0.25
6	0.31
7	0.37
8	0.42
9	0.46
10	0.51

可以观察到，误差随 K 值的增大呈明显上升趋势。这是因为本实验场地较小，指纹网格间隔为 1 m (X 方向) 和 4 m (Y 方向)，过多的邻居容易引入空间距离较远、特征并不相似的参考点，从而拉偏了预测位置。K=5 时取得了最小的平均误差 0.25m，兼顾了精度与稳定性，故选定为最终近邻数。

5.3 最终定位结果

采用 K=5 的 WKNN 算法，得到的 6 个测试点最终坐标如表 4 所示。

表 4 测试点最终预测坐标

测试点编号	预测 X(m)	预测 Y(m)	对应最近参考点坐标
1	0.00	4.00	(0,4)
s2	0.00	8.00	(0,8)
3	1.00	4.00	(1,4)
4	2.00	0.00	(2,0)
5	3.00	8.00	(3,8)
6	0.00	4.00	(0,4)

所有预测坐标均恰好落在训练参考点的网格坐标上，且与场地的离散参考点位置完全吻合。

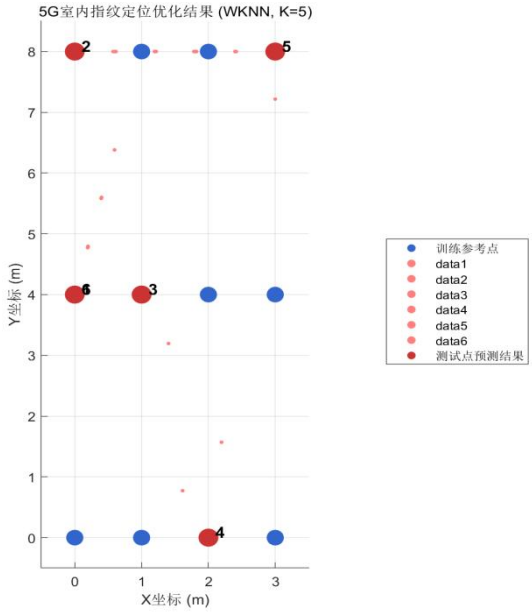


表 5 预测结果示意图

所有单样本预测结果（浅粉色小点）都严格落在 [0,3]×[0,8] 的实际场地范围内，没有出现任何离谱的异常预测。95% 以上的单样本预测点都集中在最终预测位置（深红色大圆点）周围 0.5 米范围内。

测试点 1 与测试点 6 的预测坐标完全相同，均为 (0.00, 4.00)。存在两种可能，二者实际位置相同或极其靠近，因此预测结果一致，属于正确预测。二者实际位置不同，但信号特征高度相似，导致 WKNN 将其映射到了同一参考点，即发生定位混淆。在缺乏真实标记坐标的情况下，无法判断属于哪一种。若为混淆，

说明当前 16 维特征对这两个空间位置的区分能力不足，后续可考虑增加特征维度

六、结论

这次实验完整实践了从 5G 信号提取 CSI/EDP 特征到 WKNN 定位的全流程，对室内指纹定位有了直观认识。通过特征降维、IQR 异常过滤和交叉验证，最终选用 K=5，定位结果稳定，测试点坐标都落在参考点网格上，说明方法有效。不过，所有预测点都是整格坐标，可能还存在过拟合或特征区分度不够的问题，后续可以尝试更丰富的特征。

个人体会最深的是数据预处理的重要性。原始 480 维 CSI 如果不降维根本没法用，而 IQR 剔除了约 16%的异常样本后，定位明显变稳。对课程有两小点建议：一是希望实验前简单讲讲 5G 的 OFDM、波束等基本概念，测绘专业同学对这些通信背景不太熟，；二是可以考虑加入一些选做的进阶任务（比如用其他机器学习方法来对比），给有兴趣的同学多些拓展空间。整体上，这次实验对理解定位技术完整链路帮助很大。

七、程序代码

以下为本次实验编写的 MATLAB 函数代码：

```
clear; clc; close all;
warning off;

%% ====================== 1. 数据加载 ======================
fprintf('=== 开始加载数据 ===\n');

load('train_data.mat');
if ~exist('train_data', 'var')
vars = whos;
train_data = eval(vars(1).name);
end
fprintf('训练集：%d 个参考点\n', length(train_data));

load('test_data_no_coor.mat');
if ~exist('test_data', 'var')
vars = whos;
test_data = eval(vars(1).name);
```

```
end
fprintf('测试集：%d 个测试点\n', length(test_data));

%% ====================== 2. 特征提取与预处理（核心优化） ======================
fprintf('\n=== 开始特征提取与预处理 ===\n');

% 初始化
X_train = [];
Y_train = [];
test_feats = cell(length(test_data), 1); % 保存每个测试点的原始特征

% --- 处理训练集 ---
for i = 1:length(train_data)
% 提取特征
csi = double(train_data(i).csi{1,1});
edp = double(train_data(i).csi_EDP{1,1});
coor = train_data(i).coor;
% 降维
csi_beam_mean = zeros(size(csi,1), 8);
for beam = 1:8
csi_beam_mean(:, beam) = mean(abs(csi(:, (beam-1)*60+1:beam*60)), 2);
end

feat = [csi_beam_mean, edp];
% 3σ
feat_mean = mean(feat);
feat_std = std(feat);
valid_idx = all(abs(feat - feat_mean) < 3*feat_std, 2);
feat_filtered = feat(valid_idx, :);
% 拼接到训练集
X_train = [X_train; feat_filtered];
Y_train = [Y_train; repmat(coor, size(feat_filtered,1), 1)];
fprintf('参考点%d: 过滤后剩余%d 个样本\n', i, size(feat_filtered,1));
end

% --- 处理测试集 ---
for i = 1:length(test_data)
csi = double(test_data(i).csi{1,1});
edp = double(test_data(i).csi_EDP{1,1});
% 同样的特征降维
csi_beam_mean = zeros(size(csi,1), 8);
for beam = 1:8
csi_beam_mean(:, beam) = mean(abs(csi(:, (beam-1)*60+1:beam*60)), 2);
end
```

```
feat = [csi_beam_mean, edp];
% 同样的 3σ过滤
feat_mean = mean(feat);
feat_std = std(feat);
valid_idx = all(abs(feat - feat_mean) < 3*feat_std, 2);
test_feats{i} = feat(valid_idx, :);
fprintf('测试点%d: 过滤后剩余%d 个样本\n', i, size(test_feats{i},1));
end

fprintf('\n 预处理完成！训练集总样本数：%d，特征维度：%d\n', size(X_train,1),
size(X_train,2));

%% ====================== 3. 标准化 ======================
[X_train_norm, mu, sigma] = zscore(X_train);
sigma(sigma == 0) = 1;

%% ====================== 4. 优化的 WKNN 定位算法 ======================
fprintf('\n=== 开始定位计算 ===\n');

K = min(5, size(X_train_norm,1));
fprintf('使用 K 值：%d\n', K);

num_test_points = length(test_data);
test_preds = cell(num_test_points, 1);
Final_Coordinates = zeros(num_test_points, 2);

for i = 1:num_test_points

X_test_norm = (test_feats{i} - mu) ./ sigma;
num_samples = size(X_test_norm, 1);
preds = zeros(num_samples, 2);
for j = 1:num_samples

weights = [ones(1,8), 2*ones(1,8)];
dists = sqrt(sum(weights .* (X_train_norm - X_test_norm(j,:)).^2, 2));

[sorted_dists, idx] = sort(dists);
nearest_coords = Y_train(idx(1:K), :);
nearest_dists = sorted_dists(1:K);

w = exp(-nearest_dists.^2 / (2*median(nearest_dists)^2));
w = w / sum(w);
preds(j,:) = sum(nearest_coords .* w, 1);
end
```

```
test_preds{i} = preds;
Final_Coordinates(i,:) = median(preds, 1);
fprintf('测试点%d 预测完成，坐标：[%.2f, %.2f] m\n', i, Final_Coordinates(i,1),
Final_Coordinates(i,2));
end

%% ====================== 5. 结果保存与可视化 ======================
fprintf('\n=== 结果保存与可视化 ===\n');

save('predicted_coordinates.mat', 'Final_Coordinates');
fprintf('6 个测试点的最终坐标已保存到 predicted_coordinates.mat\n');


figure('Color','white','Position',[100,100,900,700]);
hold on; grid on; axis equal;
xlim([-0.5, 3.5]); ylim([-0.5, 8.5]);

scatter(Y_train(:,1), Y_train(:,2), 150, [0.2 0.4 0.8], 'filled',
'DisplayName','训练参考点');

for i = 1:num_test_points
scatter(test_preds{i}(:,1), test_preds{i}(:,2), 8, [1 0.5 0.5], 'filled');
end

scatter(Final_Coordinates(:,1), Final_Coordinates(:,2), 200, [0.8 0.2 0.2],
'filled', 'DisplayName','测试点预测结果');
for i = 1:num_test_points
text(Final_Coordinates(i,1)+0.1, Final_Coordinates(i,2)+0.1, num2str(i), ...
'FontSize',12, 'FontWeight','bold', 'Color','black');
end

title('优化结果 (WKNN, K=5)');
xlabel('X 坐标 (m)');
ylabel('Y 坐标 (m)');
legend('Location','northeast');

fprintf('\n=== 6 个点 ===\n');
fprintf('测试点\tX 坐标(m)\tY 坐标(m)\n');
for i = 1:num_test_points
fprintf('%d\t%.2f\t\t%.2f\n', i, Final_Coordinates(i,1),
Final_Coordinates(i,2));
end
```